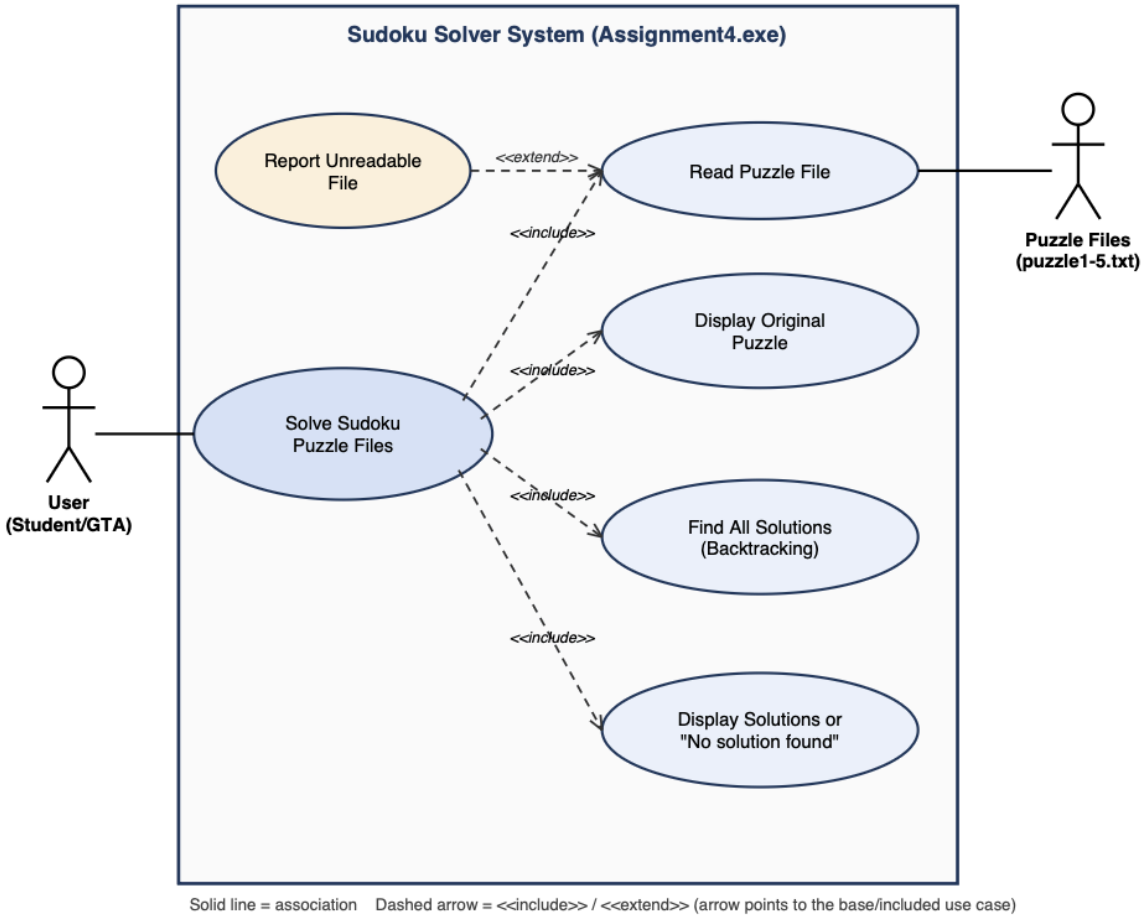
UML Use Case Details: Sudoku Solver System

1. Use Case Diagram



a. Identify / Discover a Use Case

- Use Case Name: Solve Sudoku Puzzles
- Primary Actor: User / Student
- Secondary Actor: File System (Sudoku text files puzzle1.txt through puzzle5.txt)
- Goal: Successfully parse, validate, and solve a batch of partially completed 9x9 Sudoku puzzles using recursive backtracking, and output all valid solutions or report if no solution exists to the user via the console.

b. Provide a Brief Description for the Use Case

The user runs the program, and for each of the five puzzle files (puzzle1.txt through puzzle5.txt) the system loads the 9x9 grid, prints the file name and the original puzzle, uses recursive depth-first search

with backtracking to find every valid solution, and prints each solution, or "No solution found" if none exists.

c. Provide a step-by-step flow (outline) of the events for the use case

1. User runs Assignment4.exe on the EECS cycle server from the folder containing puzzle1.txt - puzzle5.txt.
2. System begins with the first puzzle file (puzzle1.txt).
3. System reads the file into a 9x9 grid (digits 1-9 are givens, '_' is a blank cell).
4. System prints the puzzle file name and the original puzzle.
5. System finds the next empty cell.
6. System computes the candidate digits for that cell (digits not already in its row, column, or 3x3 block).
7. If there are no candidates, the system backtracks to the previous cell and tries its next candidate.
8. Otherwise, the system places a candidate and recursively repeats steps 5-7 on the remaining empty cells.
9. When no empty cells remain, the system records the completed grid as a solution, then backtracks to keep searching for additional solutions.
10. When all possibilities are exhausted, the system prints every solution found, or "No solution found" if there were none.
11. The system repeats steps 3-10 for puzzle2.txt through puzzle5.txt.
12. After the fifth puzzle, the program ends

d. Provide a detailed description for the use case

Use Case Name: Solve Sudoku Puzzle Files
Primary Actor: User (student or GTA running the program)
Secondary Actor: Puzzle Files (puzzle1.txt - puzzle5.txt), which supply the input data
Goal: Find and display all solutions (or report that none exist) for each of the five Sudoku puzzles.
Trigger: User launches Assignment4.exe.
Preconditions: Executable compiled and runnable on the cycle server; the five puzzle files are present in the working directory; each file holds 9 rows of 9 characters (digits 1-9 or '_').
Postconditions (success): For every puzzle, the file name, original puzzle, and all solutions (or "No solution found") have been printed. Puzzle files are unchanged.
Postconditions (failure): If a file cannot be read, an error message is shown for that file and processing continues with the remaining files.
Main Success Flow:
1. User launches the program.
2. System opens the next puzzle file and loads the grid.
3. System prints the file name and the original puzzle.
4. System picks the first empty cell and builds its candidate list from row, column, and block constraints.
5. System places a candidate and recurses on the next empty cell.
6. When the grid is full, System stores it as a solution and backtracks to look for more.
7. System prints each stored solution.
8. System repeats for the remaining files, then exits.

Alternate Flow A: No solution: At step 6, if the search ends with zero stored solutions (including puzzles whose givens already conflict), System prints "No solution found" in place of a solution and moves to the next file.
Alternate Flow B: Multiple solutions: At step 7, if more than one solution was stored, System prints all of them, each clearly separated.
Extension: Report Unreadable File: At step 2, if the file is missing or cannot be opened, System prints an error naming the file, skips it, and continues with the next file.
Business Rules: Each digit 1-9 appears exactly once in every row, every column, and every 3x3 block. Search must be brute-force depth-first with backtracking, implemented recursively and with objects (not free functions). All solutions must be found, not just the first.
Special Requirements: Single source file Assignment4.cpp; must compile and run on the EECS cycle servers; each puzzle should finish in well under a second; code must carry prologue and per-line comments per the rubric.
Includes: Read Puzzle File; Display Original Puzzle; Find All Solutions (Backtracking); Display Solutions or "No solution found"
Extended by: Report Unreadable File
Assumptions: Input files are well-formed apart from the cases above; the grid is always 9x9; solutions are printed in the order the search discovers them.